

Modelos Avanzados de Computación Examen II

FACULTAD
DE
CIENCIAS
UNIVERSIDAD DE GRANADA



Los Del DGIIM, losdeldgiim.github.io

Doble Grado en Ingeniería Informática y Matemáticas
Universidad de Granada



Esta obra está bajo una Licencia Creative Commons Atribución-NoComercial-SinDerivadas 4.0 Internacional (CC BY-NC-ND 4.0).

Eres libre de compartir y redistribuir el contenido de esta obra en cualquier medio o formato, siempre y cuando des el crédito adecuado a los autores originales y no persigas fines comerciales.

Modelos Avanzados de Computación Examen II

Los Del DGIIM, losdeldgiim.github.io

José Manuel Sánchez Varbas

Granada, 2026

Asignatura Modelos Avanzados de Computación.

Curso Académico 2023-24.

Grado Grado en Ingeniería Informática.

Descripción Ordinaria.

Fecha 12 de Junio de 2024.

Ejercicio 1 (2.4 puntos). Determinar cuáles de los siguientes problemas son decidibles, semidecidibles o no semidecidibles, justificando las respuestas (se supone que las MTs tienen a $\{0, 1\}$ como alfabeto de entrada):

1. Dada una MT, M , ¿es cierto que siempre que acepta una palabra u también acepta u^{-1} ?
2. Dada una MT M , ¿es cierto que todas las palabras $|u|$ de longitud menor o igual que 100 son aceptadas en un número de pasos menor o igual a $|u|^3$?
3. Dada una MT M , ¿es cierto que no es de complejidad n^3 en tiempo?
4. Dada una MT no determinista M y una palabra u , ¿es cierto que todos los caminos que puede seguir la MT M cuando lee la palabra u terminan en menos de 100 pasos?

Ejercicio 2 (1.3 puntos). Si $P(x)$ es un problema y $C - P(x)$ es su problema contrario, enuncia y demuestra (razona por qué son ciertos) los resultados que conozcas y que relacionen la decidibilidad y semidecidibilidad de ambos problemas.

Ejercicio 3 (2.4 puntos). Razona si son verdaderas o falsas las siguientes afirmaciones:

1. Si tenemos una MT multipista, entonces se puede simular con una MT de una sola cinta con el mismo número de pasos.
2. $PESPACIO = NPESPACIO$.
3. Descomponer un número en factores primos es un problema de la clase FNPT, pero no se conoce ningún algoritmo polinómico que lo resuelva.
4. Todo programa con variables numéricas se puede simular con un programa con variables que se ejecuta con el mismo número de pasos para cualquier entrada.

Ejercicio 4 (1.3 puntos). Elige una de las siguientes cuestiones:

1. Enuncia los problemas de la partición y el problema de la mochila en versión decisión. Describe cómo se demuestra que el problema de la mochila es NP-completo sabiendo que el problema de la partición lo es.
2. En la reducción del cubrimiento por vértices al circuito hamiltoniano, describe en qué se transforma una arista del problema del cubrimiento por vértices y qué propiedades tiene la estructura en la que se transforma.

Ejercicio 5 (1.3 puntos). ¿A qué clase de complejidad básica pertenece el problema de la existencia de caminos en grafos dirigidos? Justifica la respuesta.

Ejercicio 6 (1.3 puntos). Enuncia el problema de la red feliz. Describe brevemente un algoritmo que lo resuelva, ¿a qué clase de complejidad pertenece?

Solución.

Ejercicio 1 (2.4 puntos). Determinar cuáles de los siguientes problemas son decidibles, semidecidibles o no semidecidibles, justificando las respuestas (se supone que las MTs tienen a $\{0, 1\}$ como alfabeto de entrada):

1. Dada una MT, M , ¿es cierto que siempre que acepta una palabra u también acepta u^{-1} ?

Definimos

$$P_1 = \{\langle M \rangle : \forall u \in \{0, 1\}^*, u \in L(M) \Rightarrow u^{-1} \in L(M)\}.$$

Es decir, P_1 pregunta si $L(M)$ es cerrado por inversión. No es decidible por el Teorema de Rice, ya que es una propiedad que depende solo de $L(M)$ y no de la máquina concreta M . Además, es no trivial: $\emptyset$ la cumple, pero $\{01\}$ no la cumple, pues $01 \in \{01\}$ pero $(01)^{-1} = 10 \notin \{01\}$.

Veamos que no es semidecidible. Reducimos desde

$$\text{DIAGONAL}(M) = \{\langle M \rangle : M \text{ no acepta } \langle M \rangle\},$$

que no es semidecidible. Dada una MT M , construimos una MT M' así:

Algoritmo 1 MT M'

Entrada: $x \in \{0, 1\}^*$
 Simular M con entrada $\langle M \rangle$.
Si M acepta $\langle M \rangle$ **entonces**
 Si $x = 01$ **entonces**
 Aceptar.
 Si no
 Rechazar.
 Fin Si
Fin Si

Si M no acepta $\langle M \rangle$, entonces $L(M') = \emptyset$, que sí es cerrado por inversión. Por tanto, $\langle M' \rangle \in P_1$.

Si M acepta $\langle M \rangle$, entonces $L(M') = \{01\}$, que no es cerrado por inversión, porque $10 \notin L(M')$. Por tanto, $\langle M' \rangle \notin P_1$.

Así,

$$\langle M \rangle \in \text{DIAGONAL} \iff \langle M' \rangle \in P_1.$$

Por tanto, $\text{DIAGONAL} \propto P_1$. Como DIAGONAL no es semidecidible, P_1 no es semidecidible.

2. Dada una MT M , ¿es cierto que todas las palabras $|u|$ de longitud menor o igual que 100 son aceptadas en un número de pasos menor o igual a $|u|^3$?

Este problema es decidible. En efecto, el conjunto

$$A = \{u \in \{0, 1\}^* : |u| \leq 100\}$$

es finito. Basta simular M sobre cada palabra $u \in A$ durante, como máximo, $|u|^3$ pasos.

Algoritmo 2 Decisor

Entrada: Una codificación $\langle M \rangle$ de una MT M
Para cada palabra $u \in \{0, 1\}^*$ con $|u| \leq 100$ **hacer**
 Simular M con entrada u durante $|u|^3$ pasos.
 Si M no acepta u en esos $|u|^3$ pasos **entonces**
 Rechazar.
 Fin Si
Fin Para
Aceptar.

El algoritmo siempre termina, porque hay un número finito de palabras u con $|u| \leq 100$ y cada simulación dura un número finito de pasos. Por tanto, el problema es decidible.

3. Dada una MT M , ¿es cierto que no es de complejidad n^3 en tiempo?

Definimos

$$P_3 = \{\langle M \rangle : M \text{ no tiene complejidad temporal } n^3\}.$$

Es decir, $\langle M \rangle \in P$ si existe alguna entrada u tal que M no termina en, como máximo, $|u|^3$ pasos.

El problema es semidecidible. Basta buscar una palabra testigo.

Algoritmo 3 MTND semidecisora de P

Entrada: Una codificación $\langle M \rangle$ de una MT M
Selecciona de forma no determinista una palabra $u \in \{0, 1\}^*$.
Simula M con entrada u durante $|u|^3$ pasos.
Si M no termina en esos $|u|^3$ pasos **entonces**
 Aceptar.
Si no
 Rechazar.
Fin Si

Si M no tiene complejidad temporal n^3 , entonces existe una palabra u tal que M no termina en $|u|^3$ pasos. La rama no determinista que escoge precisamente esa palabra u acepta.

Si M sí tiene complejidad temporal n^3 , entonces para toda palabra u , M termina en, como máximo, $|u|^3$ pasos. Por tanto, ninguna rama acepta.

Luego P_3 es semidecidible.

Veamos que no es decidible. Reducimos desde PARADA. Dada una instancia $\langle N, w \rangle$, construimos una MT M' que, sobre entrada x , funciona así:

Algoritmo 4 MT M'

Entrada: $x \in \{0, 1\}^*$

Simular N con entrada w durante $|x|$ pasos.

Si N ha parado con entrada w en esos $|x|$ pasos **entonces**

Ejecutar un bucle de $|x|^4$ pasos y parar.

Si no

Parar inmediatamente.

Fin Si

Si N no para con entrada w , entonces M' siempre para rápidamente, luego sí tiene complejidad temporal n^3 .

Si N para con entrada w en T pasos, entonces para toda entrada x con $|x| \geq T$, la máquina M' detecta la parada de $N(w)$ y ejecuta $|x|^4$ pasos. Por tanto, M' no tiene complejidad temporal n^3 .

Así,

$$\langle N, w \rangle \in \text{PARADA} \iff \langle M' \rangle \in P_3.$$

Por tanto, $\text{PARADA} \propto P_3$. Si P_3 fuera decidible, también lo sería PARADA, contradicción. Luego P_3 no es decidible.

En conclusión, el problema es semidecidible pero no decidible.

4. Dada una MT no determinista M y una palabra u , ¿es cierto que todos los caminos que puede seguir la MT M cuando lee la palabra u terminan en menos de 100 pasos?

Este problema es decidable. Como la cota 100 es fija, basta explorar todos los caminos de cómputo posibles de M con entrada u hasta profundidad 100. Como una MT no determinista tiene un número finito de transiciones posibles en cada configuración, el árbol de cómputo de profundidad 100 es finito.

Algoritmo 5 Decisor

Entrada: Una MT no determinista M y una palabra u

Construir todas las configuraciones alcanzables desde la configuración inicial en menos de 100 pasos.

Si toda rama ha terminado antes de llegar al paso 100 **entonces**

 Aceptar.

Si no

 Rechazar.

Fin Si

El algoritmo siempre termina porque solo explora un árbol finito. Por tanto, el problema es decidable.

Ejercicio 2 (1.3 puntos). Si $P(x)$ es un problema y $C - P(x)$ es su problema contrario, enuncia y demuestra (razona por qué son ciertos) los resultados que conozcas y que relacionen la decidibilidad y semidecidibilidad de ambos problemas.

Sea $P(x)$ un problema de decisión y sea $C - P(x)$ su problema contrario, es decir,

$$P(x) = \text{SI} \iff C - P(x) = \text{NO}.$$

Equivalentemente, si $L(P)$ es el lenguaje asociado a P , entonces

$$L(C - P) = \overline{L(P)}.$$

Veamos los resultados que relacionan decidibilidad y semidecidibilidad.

1. P es decidable si y solo si $C - P$ es decidable.

En efecto, si P es decidable, existe una MT que siempre termina y responde correctamente a P . Para decidir $C - P$, basta ejecutar ese decisor y cambiar la respuesta: si responde SI, respondemos NO; si responde NO, respondemos SI. Como el decisor de P siempre termina, el de $C - P$ también.

El recíproco es igual, intercambiando los papeles de P y $C - P$.

2. Si P es decidable, entonces P y $C - P$ son semidecidibles.

En efecto, todo problema decidable es semidecidible: basta usar el decisor y aceptar cuando la respuesta sea SI. Por el apartado anterior, si P es decidable, también lo es $C - P$, luego ambos son semidecidibles.

3. P es decidable si y solo si P y $C - P$ son semidecidibles.

Si P es decidable, ya hemos visto que ambos son semidecidibles.

Recíprocamente, supongamos que P y $C - P$ son semidecidibles. Entonces existen dos MT M_1 y M_2 tales que M_1 acepta los casos positivos de P y M_2 acepta los casos positivos de $C - P$, es decir, los casos negativos de P .

Para decidir P sobre una entrada x , ejecutamos en paralelo $M_1(x)$ y $M_2(x)$. Si M_1 acepta, respondemos SI. Si M_2 acepta, respondemos NO. Como toda entrada x pertenece a $L(P)$ o a $\overline{L(P)}$, una de las dos máquinas aceptará en tiempo finito. Por tanto, este procedimiento siempre termina y decide P .

4. Si P es semidecidible pero no decidable, entonces $C - P$ no es semidecidible.

En efecto, si $C - P$ también fuera semidecidible, entonces por el apartado anterior P sería decidable, contradicción. Luego $C - P$ no puede ser semidecidible.

Ejercicio 3 (2.4 puntos). Razona si son verdaderas o falsas las siguientes afirmaciones:

1. Si tenemos una MT multipista, entonces se puede simular con una MT de una sola cinta con el mismo número de pasos.

Verdadera. Una MT multipista no tiene más de una cinta, sino una sola cinta dividida en varias pistas. Cada casilla contiene una tupla de símbolos, uno por pista. Por tanto, una MT ordinaria puede simularla tomando como alfabeto de cinta el producto de los alfabetos de las pistas. Así, cada paso de la MT multipista se simula por un solo paso de la MT de una cinta.

2. $\text{PESPACIO} = \text{NPESPACIO}$.

Verdadera. Es el Teorema de Savitch. En particular, para funciones constructibles en espacio $s(n) \geq \log n$,

$$\text{NPESPACIO}(s(n)) \subseteq \text{ESPACIO}(s(n)^2).$$

Tomando $s(n)$ polinómica, se obtiene

$$\text{NPESPACIO} \subseteq \text{PESPACIO}.$$

La otra inclusión es inmediata, porque toda MT determinista es un caso particular de MT no determinista. Luego

$$\text{PESPACIO} = \text{NPESPACIO}.$$

3. Descomponer un número en factores primos es un problema de la clase FNPT , pero no se conoce ningún algoritmo polinómico que lo resuelva.

Verdadera. Es un problema de **FNPT**: para todo número p existe una descomposición en factores primos, y una solución candidata se verifica en tiempo polinómico comprobando el producto y los certificados de primalidad de los factores. Por tanto es un problema total de FNP . Sin embargo, no se conoce un algoritmo clásico determinista en tiempo polinómico que lo resuelva, es decir, no se sabe si pertenece a **FP**.

4. Todo programa con variables numéricas se puede simular con un programa con variables que se ejecuta con el mismo número de pasos para cualquier entrada.

Falsa. Por la tesis de Church-Turing (actualmente aceptada como cierta), los programas con variables numéricas y los programas con variables son equivalentes en poder de cálculo, es decir, pueden simularse mutuamente. Pero la simulación no tiene por qué conservar el mismo número de pasos. Codificar números como palabras y simular operaciones aritméticas puede requerir varios pasos por cada paso del programa original.

Ejercicio 4 (1.3 puntos). Elige una de las siguientes cuestiones:

1. Enuncia los problemas de la partición y el problema de la mochila en versión decisión. Describe cómo se demuestra que el problema de la mochila es NP-completo sabiendo que el problema de la partición lo es.

PARTICIÓN. Datos: un conjunto finito C y una función $s : C \rightarrow \mathbb{N}$. Pregunta: ¿existe $C' \subseteq C$ tal que

$$\sum_{c \in C'} s(c) = \sum_{c \in C \setminus C'} s(c)?$$

MOCHILA. Datos: un conjunto finito de objetos U , un tamaño $s(u) \in \mathbb{N}$, un valor $v(u) \in \mathbb{N}$ para cada $u \in U$, una cota de tamaño B y una cota de valor K . Pregunta: ¿existe $U' \subseteq U$ tal que

$$\sum_{u \in U'} s(u) \leq B \quad \text{y} \quad \sum_{u \in U'} v(u) \geq K?$$

Veamos que MOCHILA es NP-completo sabiendo que PARTICIÓN lo es.

Primero, MOCHILA está en NP: dado un subconjunto $U' \subseteq U$, se comprueba en tiempo polinómico si satisface las dos desigualdades.

Reducimos PARTICIÓN a MOCHILA. Dada una instancia (C, s) de PARTICIÓN, construimos la instancia de MOCHILA:

$$U = C, \quad s'(u) = s(u), \quad v'(u) = s(u), \quad B = K = \frac{1}{2} \sum_{u \in C} s(u).$$

Entonces existe una partición de C en dos partes con la misma suma si y solo si existe $U' \subseteq U$ tal que

$$\sum_{u \in U'} s'(u) \leq B \quad \text{y} \quad \sum_{u \in U'} v'(u) \geq K.$$

Como $s' = v'$ y $B = K$, ambas desigualdades obligan a

$$\sum_{u \in U'} s(u) = \frac{1}{2} \sum_{u \in C} s(u).$$

Por tanto, PARTICIÓN $\propto$ MOCHILA. Como PARTICIÓN es NP-completo y MOCHILA está en NP, MOCHILA es NP-completo.

2. En la reducción del cubrimiento por vértices al circuito hamiltoniano, describe en qué se transforma una arista del problema del cubrimiento por vértices y qué propiedades tiene la estructura en la que se transforma.

En la reducción de **Cubrimiento por Vértices** a **Circuito Hamiltoniano**, cada arista $e = \{u, v\}$ del grafo original se sustituye por un subgrafo, o *componente*, asociado a esa arista.

Esta componente tiene dos partes simétricas: una correspondiente al extremo u y otra correspondiente al extremo v . Intuitivamente, el circuito hamiltoniano podrá recorrer la componente entrando por el lado de u , por el lado de v , o por ambos.

Las propiedades esenciales de esta estructura son:

- Si solo uno de los extremos, por ejemplo u , está en el cubrimiento, entonces el circuito puede entrar por el lado de u y recorrer toda la componente asociada a la arista e .
- Si los dos extremos u y v están en el cubrimiento, entonces el circuito puede recorrer una parte de la componente desde el lado de u y la otra desde el lado de v .
- Si ninguno de los extremos u ni v está en el cubrimiento, entonces la componente no puede ser recorrida correctamente por el circuito hamiltoniano.
- Por tanto, que el circuito hamiltoniano pueda pasar por todas las componentes equivale a que cada arista tenga al menos un extremo elegido, que es precisamente la condición de cubrimiento por vértices.

Así, las aristas del grafo original se transforman en componentes que fuerzan que, para poder construir un circuito hamiltoniano, cada arista original esté cubierta por al menos uno de sus dos extremos.

Ejercicio 5 (1.3 puntos). ¿A qué clase de complejidad básica pertenece el problema de la existencia de caminos en grafos dirigidos? Justifica la respuesta.

El problema de la existencia de caminos en grafos dirigidos pertenece a la clase **NL**.

Dado un grafo dirigido $G = (V, E)$ y dos nodos $s, t \in V$, queremos decidir si existe un camino dirigido de s a t .

Está en **NL** porque una MT no determinista puede hacer lo siguiente:

Algoritmo 6 MTND para accesibilidad en grafos dirigidos

Entrada: Un grafo dirigido $G = (V, E)$ y dos nodos s, t

$v \leftarrow s$

Para $i = 1, \dots, |V|$ **hacer**

Si $v = t$ **entonces**

 Aceptar.

Fin Si

 Elegir no determinísticamente un sucesor w de v .

$v \leftarrow w$.

Fin Para

Rechazar.

Solo necesita guardar el nodo actual v , el contador i y los nodos s, t , todo ello usando espacio $O(\log |V|)$. Por tanto, el problema está en **NL**.

Además, es un problema básico **NL-completo**: es el problema clásico de accesibilidad en grafos dirigidos.

Ejercicio 6 (1.3 puntos). Enuncia el problema de la red feliz. Describe brevemente un algoritmo que lo resuelva, ¿a qué clase de complejidad pertenece?

El problema de la **red feliz** se enuncia así:

Datos: un grafo $G = (V, E)$ y un peso $w(i, j) \in \mathbb{Z}$ para cada arco $(i, j) \in E$.
Un **estado** es una aplicación $s : V \rightarrow \{-1, 1\}$. Un nodo i es feliz si

$$s(i) \sum_{(i,j) \in E} s(j)w(i, j) \geq 0.$$

Solución: encontrar un estado s en el que todos los nodos sean felices.
Para resolverlo, se considera la función

$$\Phi(s) = \sum_{(i,j) \in E} s(i)s(j)w(i, j).$$

Si hay un nodo infeliz i , cambiamos su signo, es decir, pasamos de $s(i)$ a $-s(i)$. Con este cambio, el valor de Φ aumenta estrictamente. Como Φ no puede crecer indefinidamente, el proceso termina. Cuando termina, no queda ningún nodo infeliz, luego la red es feliz.

Algoritmo 7 Algoritmo para red feliz

Entrada: Un grafo ponderado (V, E, w)

Tomar un estado inicial cualquiera $s : V \rightarrow \{-1, 1\}$.

Mientras exista un nodo infeliz i **hacer**

 Cambiar el signo de i : $s(i) \leftarrow -s(i)$.

Fin Mientras

Devolver s .

El problema pertenece a **TFNP**: está en FNP y es total, porque siempre existe una solución. El algoritmo anterior es pseudopolinómico, pero no necesariamente polinómico.